One-to-many Object Relationships

(https://github.com/learn-co-curriculum/p3-one-to-many) (https://github.com/learn-co-curriculum/p3-one-to-many/issues/new)

Learning Goals

- Describe one-to-many relationships.
- Explain why one-to-many relationships are important.

Key Vocab

- **Class**: a bundle of data and functionality. Can be copied and modified to accomplish a wide variety of programming tasks.
- **Object**: the more common name for an instance. The two can usually be used interchangeably.
- **Object-Oriented Programming**: programming that is oriented around data (made mobile and changeable in **objects**) rather than functionality. Python is an object-oriented programming language.
- **Function**: a series of steps that create, transform, and move data.
- **Method**: a function that is defined inside of a class.

Introduction

In object-oriented programming, a one-to-many relationship refers to a relationship between two or more objects in which one object can be associated with multiple objects of another type. For instance, a teacher can have multiple students in a class, or a university can have many departments.

To represent this relationship, we can use various techniques such as association, aggregation, or composition.

Association refers to a relationship where two objects are related, but not in a way that one object owns the other. For example, a library has an association with books as it stores them, but it doesn't own them.

Aggregation is a relationship where one object is a part of another object, and the first object can exist independently. For example, a car can have many wheels, but each wheel can also be used in other cars.

Composition is a relationship where one object is made up of another object, and the first object cannot exist without the second object. For example, a house is composed of rooms, and each room is part of the house.

In this lesson, we will discuss how we can use these techniques to create one-to-many relationships.

Association

Association is a weak relationship between otherwise unrelated objects, where the objects have their own lifetime and there is no parent object.

One way one-to-many relationship

Lets look at an example of a one way one-to-many relationship. The example will use a `Shopper` and `GroceryItem` class. The `Shopper` class will have a list attribute to store the related `GroceryItem` objects.

In this case it does not make sense for the `GroceryItem` to know about the `Shopper` .

Here's an example implementation:

```
class GroceryItem:
    def __init__(self, name, price):
        self.name = name
        self.price = price

class Shopper:
    def __init__(self, name):
        self.name = name
        self.grocery_items = []
```

In this example, the `Shopper` class has a `grocery_items` attribute, which is a list of `GroceryItem` objects. Here's how you can use these classes:

```
# Create a shopper and some grocery items
shopper = Shopper("Alice")
item1 = GroceryItem("apple", 1)
item2 = GroceryItem("orange", 2)

# Add the grocery items to the shopper's list
shopper.grocery_items.append(item1)
shopper.grocery_items.append(item2)

# Print the shopper's grocery list
for item in shopper.grocery_items:
    print(item.name, item.price)

# => apple 1
# => orange 2
```

Two way one-to-many relationship

Lets look at an example of a two way one-to-many relationship. In this example we will use a `Teacher` and `Student` class. The reason we want to make this a two-way relationship is because the `Student` class may need to know who the `Teacher` is. In the previous example the `GroceryItem` has no reason to know who the shopper associated with it is.

For this example we will also add type checks to the setters and methods and raise a `TypeError` if the types are incorrect. In the setters for the attributes we can use `isinstance` to make sure the user is not passing in a string or number when the attribute expects an object.

A property decorator in Python is a built-in decorator that allows you to give special functionality to certain methods in a class, making them act as getters, setters. In this code example, the `@property` decorator is used to define a getter and setter method for the teacher attribute. We can use these methods to validate values. In `Student` class we validate that the teacher setter only accepts a `Teacher` object.

```
class Student:

    all = []

    def __init__(self, name, age):
        self.name = name
        self.age = age
        # teacher is protected because it is not a part of the constructor
        self._teacher = None
        Student.all.append(self)

    @property
    def teacher(self):
        return self._teacher

    @teacher.setter
    def teacher(self, value):
        if not isinstance(value, Teacher):
            raise TypeError("Teacher must be an instance of Teacher class")
        self._teacher = value

class Teacher:
    def __init__(self, name):
        self.name = name

    def students(self):
        return [student for student in Student.all if student.teacher == self]

    def add_student(self, student):
        if not isinstance(student, Student):
            raise TypeError("Student must be an instance of Student class")
        student.teacher = self
```

In this example a teacher can have multiple students and a student needs to know who their teacher is. The difference in this example is that when we access students through their teachers, we are referring back to the student class. This is due to a principle called **Single Source of Truth**.

Single Source of Truth (SSOT)

"Single source of truth" is a way of organizing code in Python so that there is only one place where each piece of information or code is defined. This helps make the code more consistent and easier to understand and change in the future.

For example, imagine you have a website that uses a certain color scheme. You might define the colors in a separate module or file, so that if you ever want to change the colors, you only have to change them in that one place. This ensures that all parts of the website use the same colors.

Another example might be a game where different characters have different abilities. You might define those abilities in a single module or class, so that the game is consistent across all characters.

With object relationships, there should be a single source of truth for each relationship. In one-way relationships, this is simple: "ones" store their "manys" in lists or "manys" store their "ones" as objects. In two-way relationships, we need to be a bit more clever. Let's look back at the example from above:

```
class Student:

    all = []

    def __init__(self, name, age):
        self.name = name
        self.age = age
        self._teacher = None
        Student.all.append(self)

    # teacher property

class Teacher:
```

```
    def __init__(self, name):
        self.name = name

    def students(self):
        return [student for student in Student.all if student.teacher == self]

    def add_student(self, student):
        if not isinstance(student, Student):
            raise TypeError("Student must be an instance of Student class")
        student.teacher = self
```

In order to give teachers access to their students, we *could* create a list attribute in the `Teacher` class to hold onto all of their related `Student` objects. However, this would be a second source of truth for the relationship- students already know about their teachers.

When we want to access a teacher's students, we can instead reference all of the objects of the `Student` class and search for those whose `Teacher` is `self`. When adding students, we can use the `add_student` method to ensure that the student is of the `Student` class and then maintain the single source of truth through the student by setting `student.teacher` to `self`.

> **Note: `add_student` isn't necessary to complete this relationship, but it allows us to modify the relationship with validations from either the `Student` or `Teacher` class while maintaining our SSOT.**

Overall, the idea behind a single source of truth in Python is to make code more reliable and easier to maintain by keeping everything organized in one place.

Aggregation

Aggregation is a form of association in which each object has its own life cycle, but there exists an ownership as well. Similarly to object inheritance we can call this a parent child relationship but instead of the attributes the parent contains the instance of a child. We will use a `Car` and `Engine` class for this example.

```
class Car:
    def __init__(self, engine):
        self.engine = engine

class Engine:
    def __init__(self, cylinders, fuelType):
        self.cylinders = cylinders
        self.fuelType = fuelType
```

The `Car` class takes in an engine object instead of creating the `Engine` object. This allows the `Engine` object to have its own lifecycle.

```
four_cylinder_engine = Engine(4, 'regular')
acura_tlx = Car(four_cylinder_engine)
```

If the `Car` is deleted the engine object won't be deleted. We can use the engine object in a different `Car` object.

Composition

Composition is a "part-of" relationship, where both entities are interdependent on each other. For instance, a `CPU` is a part of a `Computer`, and both are dependent on each other:

```
class CPU:
  def __init__(self, cpu_type):
    self.cpu_type = cpu_type

class Computer:
  def __init__(self, cpu_type):
    self.CPU = CPU(cpu_type)
```

This example represents a one-to-many relationship between computer and CPU because a computer can have many different types of CPUs. This is composition because the CPU is created within the `Computer` class.

Conclusion

When designing a system in object-oriented programming, classes are fundamental components that represent entities and their attributes. One-to-many relationships between classes are essential to efficiently organize relationships between entities. By creating classes and objects that accurately model real-world entities and their relationships, we can develop a scalable and reusable code structure.

For example, imagine you are designing a system to manage a library. A user can check out many books at once. By creating a one-to-many relationship between the `User` class and the `Book` class, we can efficiently manage the relationships between users and books.

Moreover, utilizing one-to-many relationships in our class design allows for greater flexibility and scalability. By creating objects that can be associated with multiple other objects, we can build more complex systems that can adapt to changing requirements and needs. For instance, a software application that manages employee schedules can be designed with a one-to-many relationship between employees and their work shifts. This approach allows the application to handle an increasing number of employees and shifts without the need for significant changes to the class structure.

In conclusion, one-to-many relationships are critical in object-oriented programming as they enable us to design efficient, scalable, and reusable code structures. By utilizing these relationships, we can accurately model the relationships between real-world entities, create more flexible and adaptable systems, and build more efficient software applications.

Resources

- **Python classes (https://docs.python.org/3/tutorial/classes.html)**